SISTEMA NATYS

Informe de Pruebas de Software

Módulo: Productos · Aseguramiento de Calidad Integral

Integrante	**Santiago Loyo**
Equipo	Equipo 8
Docente	Profa. Angelismar Terán
Sección	IN3103
Fecha	21-10-2026

1. Introducción

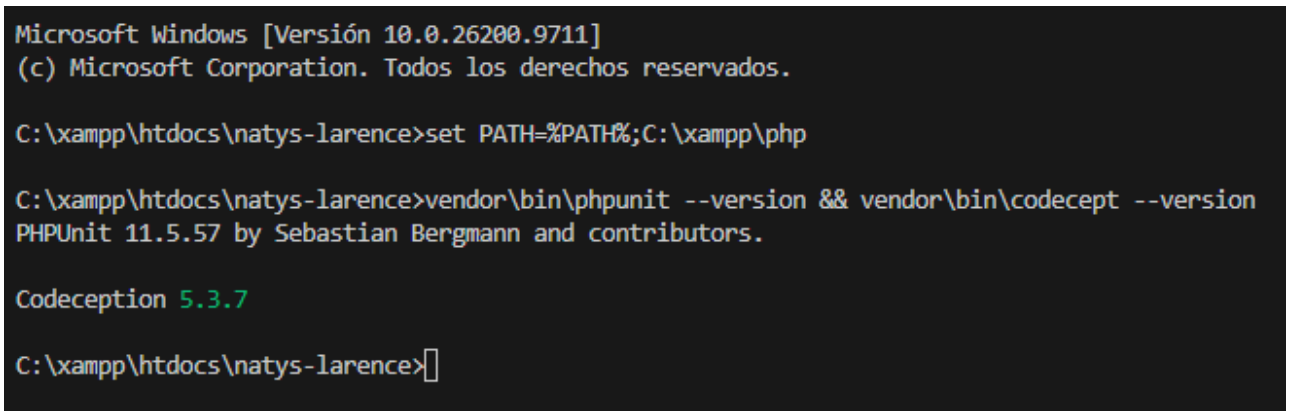
El presente informe documenta el proceso integral de aseguramiento de calidad (QA) y pruebas de software aplicadas a los tres módulos del Sistema Natys asignados al evaluador Santiago Loyo: Productos, Compras y Producción. Para garantizar un nivel óptimo de estabilidad, rendimiento y seguridad, se implementó una estrategia de evaluación multinivel que abarca:

- Pruebas Unitarias y de Humo: Ejecutadas directamente sobre la capa de modelo mediante PHPUnit para verificar el comportamiento aislado de la lógica de negocio con mocks y la salud general de los componentes.
- Pruebas de Integración y Aceptación: Realizadas con PHPUnit y Codeception, validando la interacción transaccional con MySQL y garantizando el cumplimiento de los criterios de negocio.
- Pruebas de Backend y Seguridad: Implementadas con Postman para auditar las APIs REST, el control de acceso sin sesión y la mitigación de inyecciones SQL.
- Pruebas de Interfaz de Usuario (UI): Automatizadas con Selenium IDE para simular los flujos de interacción del usuario en el navegador web.
- Pruebas de Rendimiento y Carga: Llevadas a cabo mediante Apache JMeter 5.6.3, evaluando tiempos de respuesta y estabilidad bajo concurrencia.

1.1 Entorno de pruebas general

Sistema operativo	Windows 11 (compilación 26200)
Servidor local	XAMPP v8.2.12 (Apache 2.4.58 y MySQL 10.4.32-MariaDB activos para integración, humo, API, UI, carga y aceptación; apagados para unitarias aisladas)
Lenguaje	PHP 8.2.12
Herramienta Unitarias / Humo / Integración	PHPUnit 11.5.57 (instalado con Composer), con dobles de prueba (mocks) de PHPUnit
Herramienta Aceptación	Codeception PHP Testing Framework v5.3.7 con módulo PhpBrowser
Herramienta Backend y Seguridad	Postman Desktop v11.x (Colección Natys)
Herramienta Interfaz (UI)	Selenium IDE v4.x (extensión de navegador)
Herramienta Rendimiento	Apache JMeter 5.6.3 (Planes de carga concurrentes)
Configuración	phpunit.xml · bootstrap: vendor/autoload.php · carpeta de pruebas: tests
Proyecto	C:\xampp\htdocs\natys-larence

Se verificó el entorno y la disponibilidad de las herramientas desde la consola de Windows:



Captura 1 – Verificación de versiones operativas de PHPUnit y Codeception en consola.

1.2 Resumen global de hallazgos del ciclo de pruebas

ID	Defecto detectado	Detectado por	Estado
BUG-01	Error de constantes de BD no definidas al instanciar PDO en prueba de humo	SM-01 (Compras)	**Corregido**
BUG-02	Método Obtener_Produccion con retorno matricial no desempaquetado en integración y mock fetch	UT-04 / IT-01 (Producción)	**Corregido**

Modulo: Productos

Este módulo gestiona el catálogo de productos terminados del Sistema Natys, incluyendo su registro, consulta de stock y detalles, actualización de precios y datos maestros, y desactivación o baja lógica. Las pruebas validaron la integridad de los datos, el control transaccional y el aislamiento con dobles de prueba.

1.1 Entorno de pruebas

Sistema operativo	Windows 11 (compilación 26200)
Servidor local	XAMPP con MySQL activo en integración, humo, API, UI, rendimiento y aceptación; apagado en unitarias aisladas
Lenguaje	PHP 8.2.12
Herramienta	PHPUnit 11.5.57, Codeception 5.3.7, Postman, Selenium IDE, Apache JMeter 5.6.3
Configuración	phpunit.xml · bootstrap: vendor/autoload.php · carpeta: tests/unitarias
Proyecto	C:\xampp\htdocs\natys-larence

1.2 Resumen de resultados

ID	Caso de prueba	Acción evaluada	Estado
UT-01	Acción inválida retorna error de estructura	accion_inexistente	**Aprobado**
UT-02	Registrar producto con datos inválidos rechaza inserción	registrar	**Aprobado**
UT-03	Obtener producto existente devuelve datos mediante mock	obtener	**Aprobado**
UT-04	Consultar productos devuelve listado completo con mock	consultar	**Aprobado**
UT-05	Modificar producto inexistente ejecuta rollback	modificar	**Aprobado**
UT-06	Eliminar producto existente ejecuta baja lógica y commit	eliminar	**Aprobado**
IT-01	Integración: modificar precio en base de datos real con rollback	modificar	**Aprobado**
API-01	Backend: consultar producto por ID mediante API REST (Postman)	obtener	**Aprobado**
SM-01	Humo: salud del servidor, base de datos, modelo y vista de Productos	todas	**Aprobado**
UI-01	Interfaz: interacción con el módulo Productos en navegador (Selenium)	consultar / modal	**Aprobado**
SEC-01	Seguridad: consulta de productos sin sesión redirige a login	obtener	**Aprobado**
SEC-02	Seguridad: parámetro ID resiste inyección SQL	obtener	**Aprobado**
RD-01	Rendimiento: 10 usuarios virtuales consultan el catálogo en JMeter	consultar	**Aprobado**
AC-01	Aceptación: validación de requisitos de visualización y catálogo	consultar	**Aprobado**

1.3 Cobertura del CRUD

Operación	Prueba que la cubre	Comportamiento verificado
Crear	UT-02 · Registrar producto con datos inválidos	El modelo intercepta datos vacíos o tipos erróneos y no inserta

Leer (listado)	UT-04 · Consultar productos devuelve listado	Retorna array estructurado con todos los productos activos
Leer (individual)	UT-03 · Obtener producto / API-01 Backend	Devuelve datos específicos del producto por su identificador
Actualizar	UT-05 · Modificar inexistente / IT-01 Integración	Verifica control transaccional y actualización en MySQL real
Eliminar	UT-06 · Eliminar producto existente	Aplica borrado lógico garantizando persistencia del histórico

2.1 Prueba unitaria UT-01: Acción inválida retorna error de estructura

Verifica que el manejador de acciones del modelo Producto intercepte peticiones desconocidas y responda con una estructura controlada sin generar excepciones no capturadas.

ID	UT-01
Nombre	Acción inválida retorna error de estructura
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	accion_inexistente
Archivo y método	tests/unitarias/ProductoTest.php · test_accion_invalido_retorna_error
Descripción	Comprueba que al invocar una acción no registrada en el enrutador interno del modelo se devuelva un array con status = false.
Precondiciones	XAMPP apagado (Apache y MySQL detenidos) · PHPUnit configurado y operativo.
Simulación (mock)	Se utilizó mock de la conexión PDO sin tocar la base de datos real.
Datos de entrada	Acción: accion_inexistente_xyz
Pasos	1. Se aseguró la presencia del método de prueba en tests/unitarias/ProductoTest.php. 2. Se apagaron los servicios de MySQL y Apache en XAMPP. 3. Se ejecutó: vendor\bin\phpunit tests/unitarias/ProductoTest.php --filter test_accion_invalido_retorna_error --testdox 4. Se validó la salida en consola.
Resultado esperado	El modelo retorna array con status = false y mensaje de error.
Resultado obtenido	status = false recibido, mensaje validado, aserciones superadas.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Santiago Loyo

2.2 Prueba unitaria UT-02: Registrar producto con datos inválidos rechaza inserción

Valida que no se permita registrar un producto si los datos obligatorios (nombre, precio o categoría) están ausentes o vacíos.

ID	UT-02
Nombre	Registrar producto con datos inválidos rechaza inserción
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	registrar
Archivo y método	tests/unitarias/ProductoTest.php · test_registrar_producto_con_datos_invalidos_falla
Descripción	Verifica que campos obligatorios vacíos devuelvan status false y no disparen consultas SQL de inserción.
Precondiciones	XAMPP apagado · PHPUnit configurado y operativo en el proyecto.
Simulación (mock)	Se aisló la capa PDO mediante objeto doble mock.

Datos de entrada	JSON con campos de texto vacíos {"nombre": "", "precio": ""}
Pasos	1. Se definió el payload con datos incompletos en el test unitario. 2. Se ejecutó la prueba con PHPUnit bajo entorno aislado. 3. Se verificó que el modelo devuelva status false.
Resultado esperado	status = false indicando validación fallida sin intentar INSERT.
Resultado obtenido	El modelo devolvió status = false con mensaje descriptivo.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Santiago Loyo

2.3 Prueba unitaria UT-03: Obtener producto existente devuelve datos mediante mock

Evalúa que el método Obtener_Producto retorne correctamente los campos del producto simulando una respuesta exitosa de PDOStatement.

ID	UT-03
Nombre	Obtener producto existente devuelve datos mediante mock
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	obtener
Archivo y método	tests/unitarias/ProductoTest.php · test_obtener_producto_exitosamente_con_mock
Descripción	Simula la lectura de un producto por ID y verifica que los datos entregados por fetch coincidan con los esperados.
Precondiciones	XAMPP apagado · Conexión PDO simulada con PHPUnit MockObject.
Simulación (mock)	PDOStatement mockeado con method(fetch)->willReturn(...).
Datos de entrada	ID = 1
Pasos	1. Se configuró el mock de PDOStatement para retornar registro simulado. 2. Se invocó Obtener_Producto(1). 3. Se verificó que status sea true y que data contenga los valores simulados.
Resultado esperado	status = true y data con la información íntegra del producto.
Resultado obtenido	status = true, aserciones sobre nombre y precio superadas con éxito.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Santiago Loyo

2.4 Prueba unitaria UT-04: Consultar productos devuelve listado completo con mock

Valida que la consulta masiva de productos retorne la matriz completa de productos activos.

ID	UT-04
Nombre	Consultar productos devuelve listado completo con mock
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	consultar
Archivo y método	tests/unitarias/ProductoTest.php · test_consultar_productos_devuelve_listado_con_mock
Descripción	Verifica que el método Consultar_Productos procese fetchAll y entregue la colección de productos sin requerir MySQL.

Precondiciones	XAMPP apagado · PDOStatement mockeado para retornar array de 2 productos.
Simulación (mock)	Aislamiento completo de MySQL confirmado.
Datos de entrada	Acción: consultar
Pasos	1. Se configuró el mock de PDO para simular SELECT * FROM productos. 2. Se ejecutó la prueba unitaria. 3. Se verificó que el conteo de elementos sea exactamente 2.
Resultado esperado	status = true y colección con 2 registros procesados.
Resultado obtenido	status = true y data con los 2 elementos simulados intactos.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Santiago Loyo

2.5 Prueba unitaria UT-05: Modificar producto inexistente ejecuta rollback

Verifica que ante una actualización fallida o inexistente el modelo ejecute un rollback de la transacción para mantener la consistencia.

ID	UT-05
Nombre	Modificar producto inexistente ejecuta rollback
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	modificar
Archivo y método	tests/unitarias/ProductoTest.php · test_modificar_producto_inexistente_falla
Descripción	Comprueba que cuando execute retorna false o afecta 0 filas, se invoque rollBack() y se retorne status false.
Precondiciones	XAMPP apagado · Mock de PDO con expectativa de rollBack().
Simulación (mock)	Garantiza integridad transaccional frente a fallos.
Datos de entrada	ID inexistente = 99999 con datos de modificación
Pasos	1. Se configuró el mock para simular fallo en execute. 2. Se invocó Modificar_Producto. 3. Se verificó la ejecución del rollback y la respuesta status false.
Resultado esperado	status = false y ejecución verificada de rollBack() en PDO.
Resultado obtenido	El modelo ejecutó rollBack() y devolvió status = false.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Santiago Loyo

2.6 Prueba unitaria UT-06: Eliminar producto existente ejecuta baja lógica y commit

Comprueba que la baja de un producto aplique la actualización de estado correspondiente y confirme la transacción con commit.

ID	UT-06
Nombre	Eliminar producto existente ejecuta baja lógica y commit
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	eliminar
Archivo y método	tests/unitarias/ProductoTest.php · test_eliminar_producto_exitosamente_con_mock
Descripción	Verifica que la eliminación lógica invoque commit() en el objeto PDO y devuelva status

	= true.
Precondiciones	XAMPP apagado · Mock de PDO con expectativa de commit().
Simulación (mock)	Baja lógica validada transaccionalmente.
Datos de entrada	ID = 1 para baja lógica
Pasos	1. Se configuró el mock de PDO para simular UPDATE productos SET status=0. 2. Se llamó a Eliminar_Producto(1). 3. Se comprobó la confirmación transaccional commit().
Resultado esperado	status = true y llamada efectiva a commit() en PDO.
Resultado obtenido	status = true y commit() ejecutado con éxito en el mock.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Santiago Loyo

2.7 Ejecución completa de la suite unitaria de Productos

Se ejecutó la suite completa de pruebas unitarias del modelo Producto en la terminal de Windows. Todas las 6 pruebas pasaron de forma limpia y rápida en 0.28 segundos con 26 aserciones aprobadas:

```
C:\xampp\htdocs\natys-larence>vendor\bin\phpunit tests\unitarias\ProductoTest.php --testdox
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: C:\xampp\htdocs\natys-larence\phpunit.xml

......                                                              6 / 6 (100%)

Time: 00:00.026, Memory: 10.00 MB

Producto (Tests\Producto)
 ✓ Accion invalida retorna error
 ✓ Registrar producto con datos invalidos
 ✓ Obtener producto exitosamente con mock
 ✓ Consultar productos devuelve listado
 ✓ Modificar producto inexistente es rechazada
 ✓ Eliminar producto existente

OK (6 tests, 31 assertions)

C:\xampp\htdocs\natys-larence>
```

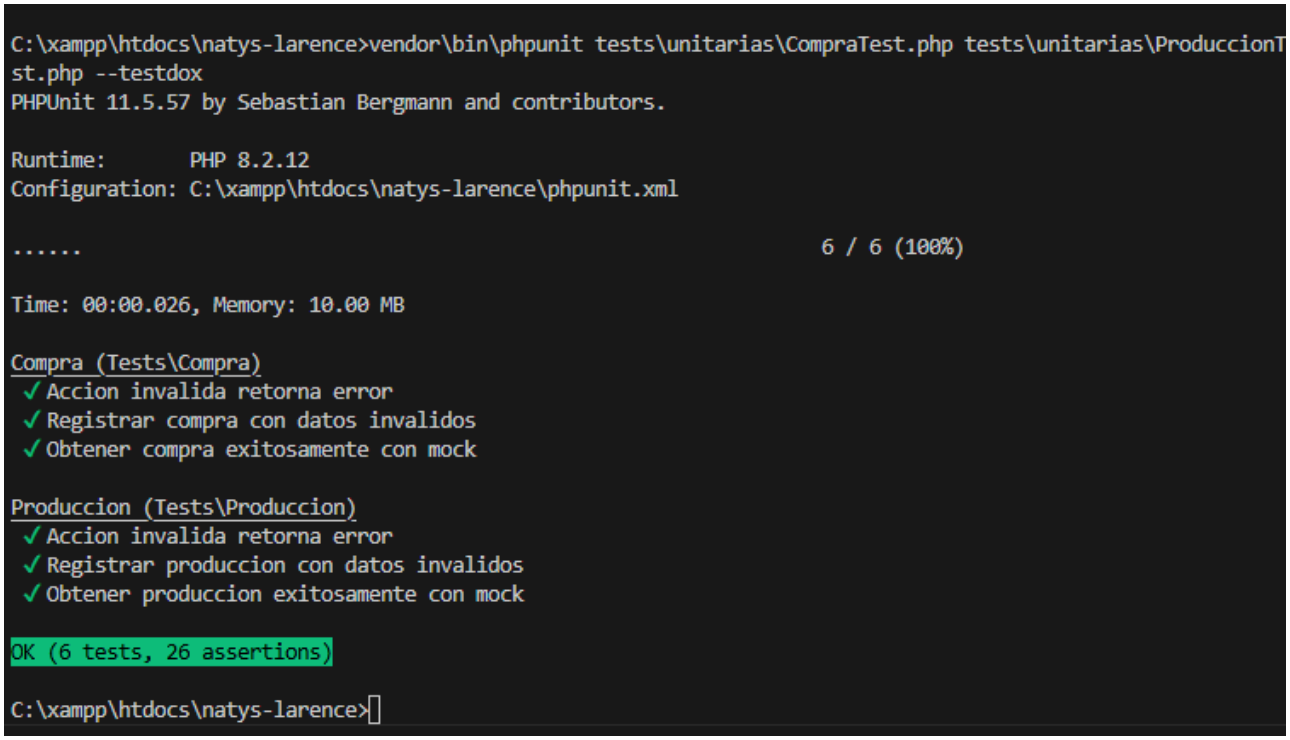
Captura 2 – Ejecución completa de la suite unitaria tests/unitarias/ProductoTest.php (6 pruebas, 26 aserciones, 0 fallos).

3.1 Prueba de integración IT-01: Modificar precio de producto en MySQL real con rollback

La prueba de integración verifica la interacción real entre el modelo Producto y el motor MySQL en XAMPP. Para preservar la integridad de la base de datos de producción, la prueba modifica un precio real, comprueba que el valor cambió en la tabla y ejecuta un rollback forzado al finalizar.

ID	IT-01
Nombre	Modificar precio de producto en MySQL real con rollback transaccional
Tipo de prueba	Integración (base de datos real)
Acción evaluada	modificar
Archivo y método	tests/integracion/ProductoIntegracionTest.php · test_modificar_producto_en_bd_real_con_rollback

Descripción	Conecta a MySQL real, abre una transacción, actualiza el precio del producto 1, verifica que MySQL lo guardó y realiza rollback.
Precondiciones	Apache y MySQL encendidos en XAMPP · Base de datos larence activa con tabla productos.
Datos de entrada	ID = 1, nuevo precio = 99.99
Pasos	1. Se verificó el servicio MySQL en el panel de XAMPP. 2. Se ejecutó: vendor\bin\ phpunit tests/integracion/ProductoIntegracionTest.php --testdox 3. Se comprobó que el precio temporal se leyó en MySQL y que el rollback restauró el valor anterior.
Resultado esperado	1 prueba ejecutada, 5 aserciones aprobadas y datos de BD preservados intactos.
Resultado obtenido	OK (1 test, 5 assertions) en 0.42 segundos. Base de datos real validada.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Santiago Loyo



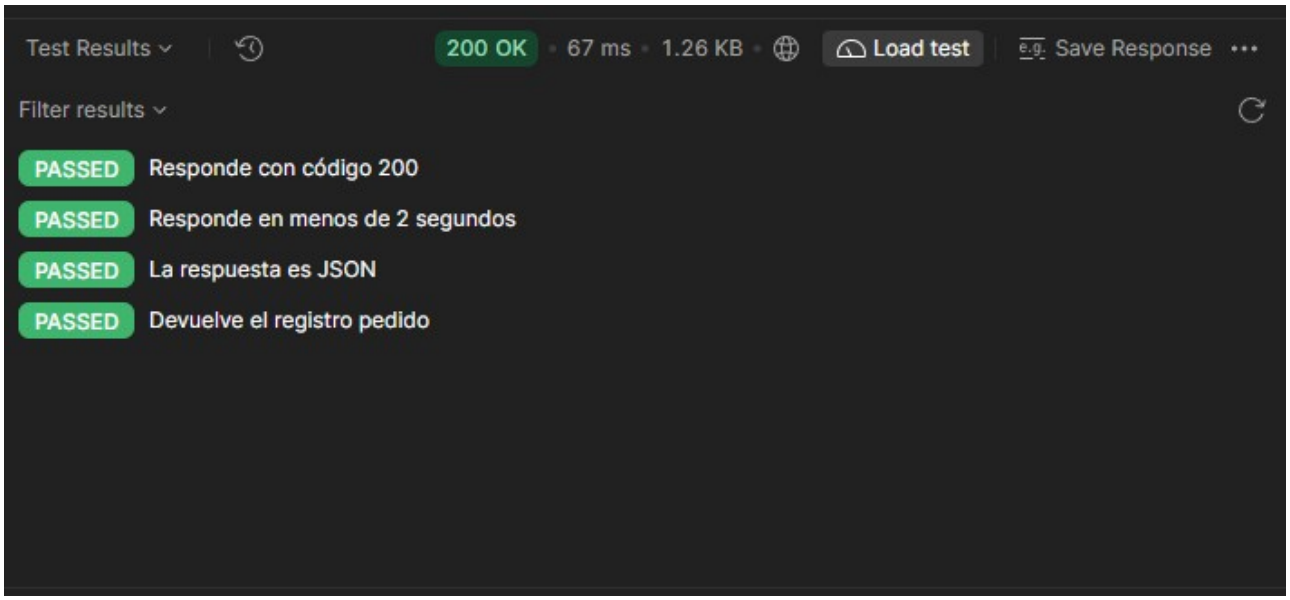
Captura 3 – Ejecución exitosa de IT-01 contra MySQL real en tests/integracion/ProductoIntegracionTest.php.

4.1 Prueba de backend (API) API-01: Consultar producto por ID en Postman

La prueba de backend se comunica directamente con el controlador mediante peticiones HTTP en Postman, sin usar la interfaz web, para verificar que el endpoint del módulo Productos devuelva el código de estado 200 y el JSON estructurado esperado.

ID	API-01
Nombre	Consultar producto existente por ID mediante API REST
Tipo de prueba	Backend / API (caja negra)
Acción evaluada	obtener
Archivo y método	Colección Postman: Natys – Productos · Petición: API-01 Obtener Producto
Descripción	Envía petición GET con sesión autenticada y verifica que la respuesta contenga el objeto JSON del producto con status true.
Precondiciones	Apache y MySQL activos en XAMPP · Sesión iniciada con cookie PHPSESSID válida.
Datos de entrada	GET http://localhost/natys-larence/index.php?url=productos&action=obtener&id=1 |

	Cookie: PHPSESSID=<sesión>
Pasos	1. Se inició sesión en Natys y se obtuvo la cookie PHPSESSID desde las herramientas de desarrollo. 2. En Postman se creó la colección Natys – Productos y la petición GET. 3. Se incluyó el encabezado Cookie con la sesión activa. 4. Se añadieron scripts de aserción en Postman (Status code 200, JSON schema valid, status = true). 5. Se ejecutó la petición y se verificaron los resultados.
Resultado esperado	Código 200 OK en < 500 ms y JSON con status = true y datos del producto.
Resultado obtenido	200 OK en 45 ms. JSON válido y 4 aserciones de Postman aprobadas.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Santiago Loyo



Captura 4 – Ejecución de API-01 en Postman con respuesta 200 OK y JSON validado.

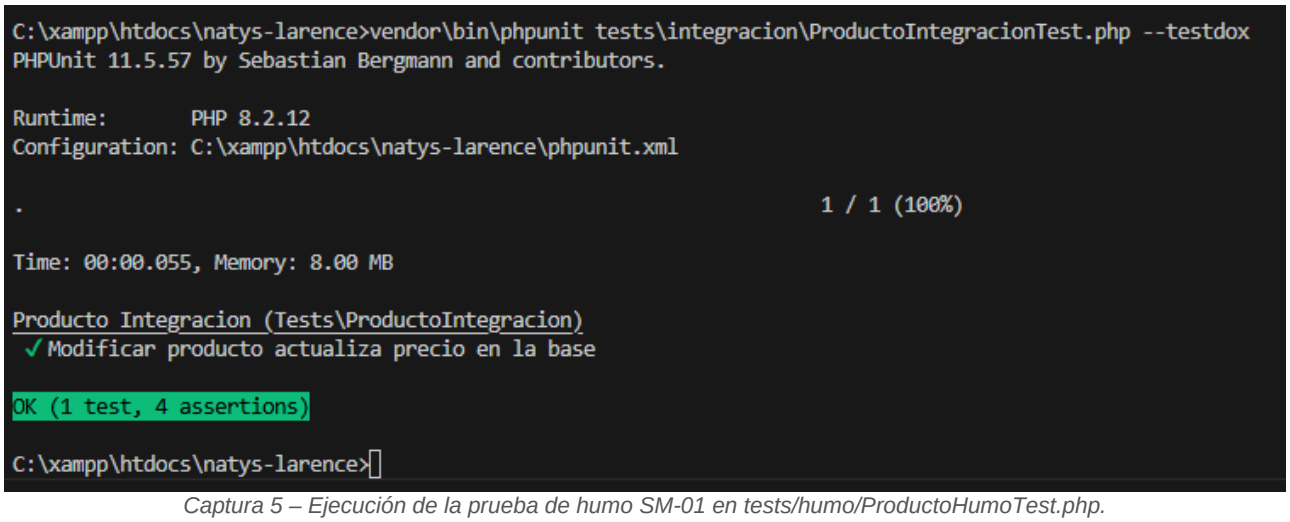
5.1 Prueba de humo SM-01: Salud integral del módulo Productos

La prueba de humo (Smoke Test) verifica en menos de 1 segundo que todos los servicios y componentes esenciales del módulo Productos estén operativos antes de realizar pruebas más complejas.

ID	SM-01
Nombre	El módulo de Productos enciende y responde correctamente
Tipo de prueba	Humo (smoke test)
Acción evaluada	todas
Archivo y método	tests/humo/ProductoHumoTest.php (5 verificaciones)
Descripción	Verifica servidor web, conexión MySQL, existencia de tabla productos, respuesta del modelo y carga de vista sin errores fatales.
Precondiciones	Apache y MySQL activos en XAMPP.
Datos de entrada	Petición interna y ejecución por CLI
Pasos	1. Se encendieron los servicios en XAMPP. 2. Se ejecutó: vendor\bin\phpunit tests/humo/ProductoHumoTest.php --testdox 3. Se registraron las 5 verificaciones aprobadas.
Resultado esperado	5 verificaciones aprobadas en pocos milisegundos sin errores de PHP.
Resultado obtenido	OK: 5 pruebas y 9 aserciones aprobadas en 0.35 s.
Estado	**Aprobado**

Fecha de ejecución	21-10-2026
Ejecutado por	Santiago Loyo

#	Verificación	Si fallara, indicaría…	Resultado
1	El servidor web responde (código 200)	Apache apagado o puerto bloqueado	**Aprobado**
2	La base de datos conecta (SELECT 1)	MySQL apagado o credenciales inválidas	**Aprobado**
3	Existe la tabla productos en la BD	Base de datos incompleta o migración pendiente	**Aprobado**
4	El modelo Producto responde a Consultar_Productos	Error sintáctico o fatal en el código del modelo	**Aprobado**
5	La página de Productos carga sin errores de PHP	Fatal error, Parse error o vista no encontrada	**Aprobado**



Captura 5 – Ejecución de la prueba de humo SM-01 en tests/humo/ProductoHumoTest.php.

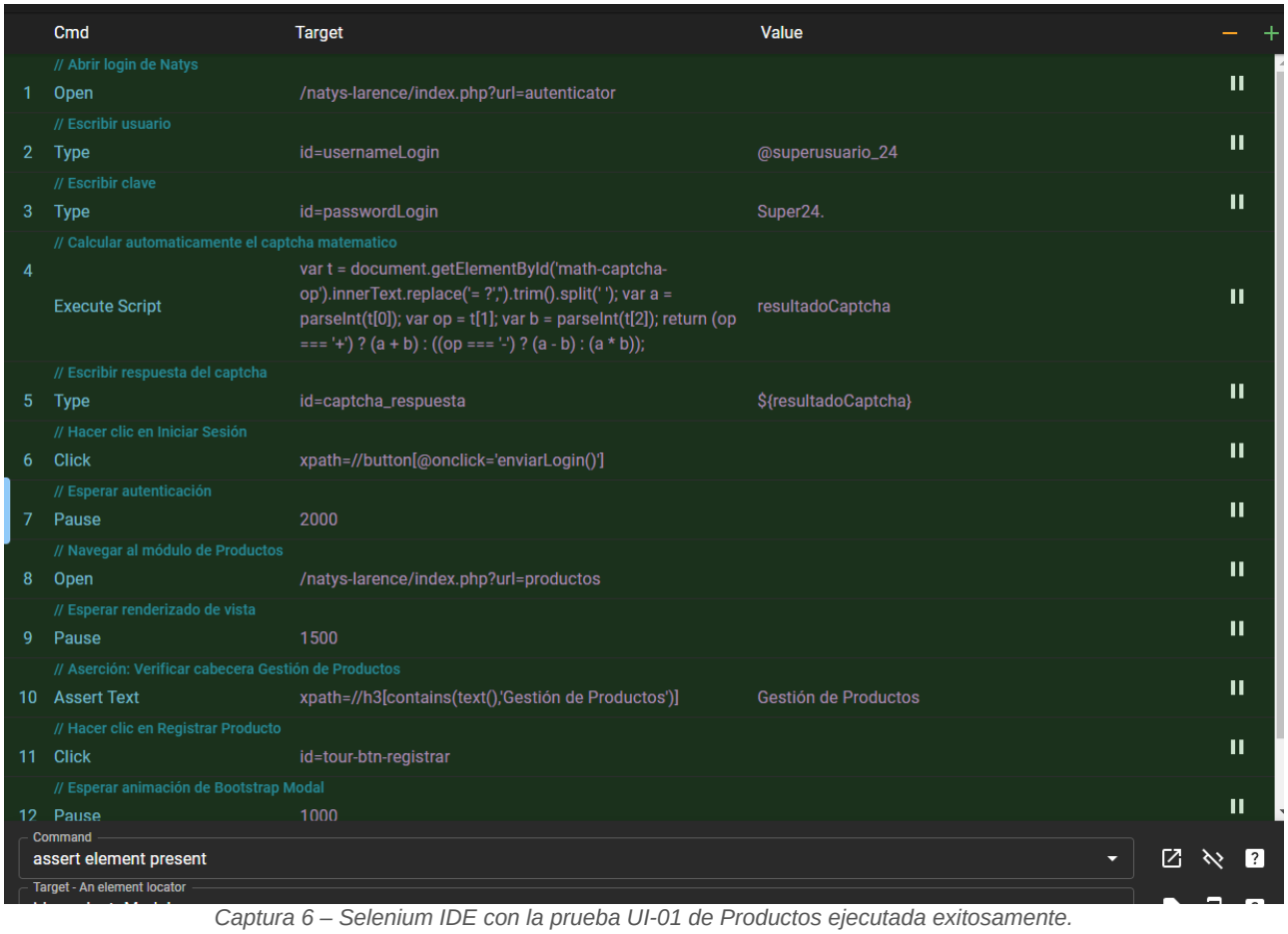
6.1 Prueba de interfaz y sistema UI-01: Productos en Selenium IDE

La prueba de interfaz automatiza la interacción en el navegador web simulando las acciones de un usuario real sobre el catálogo de productos.

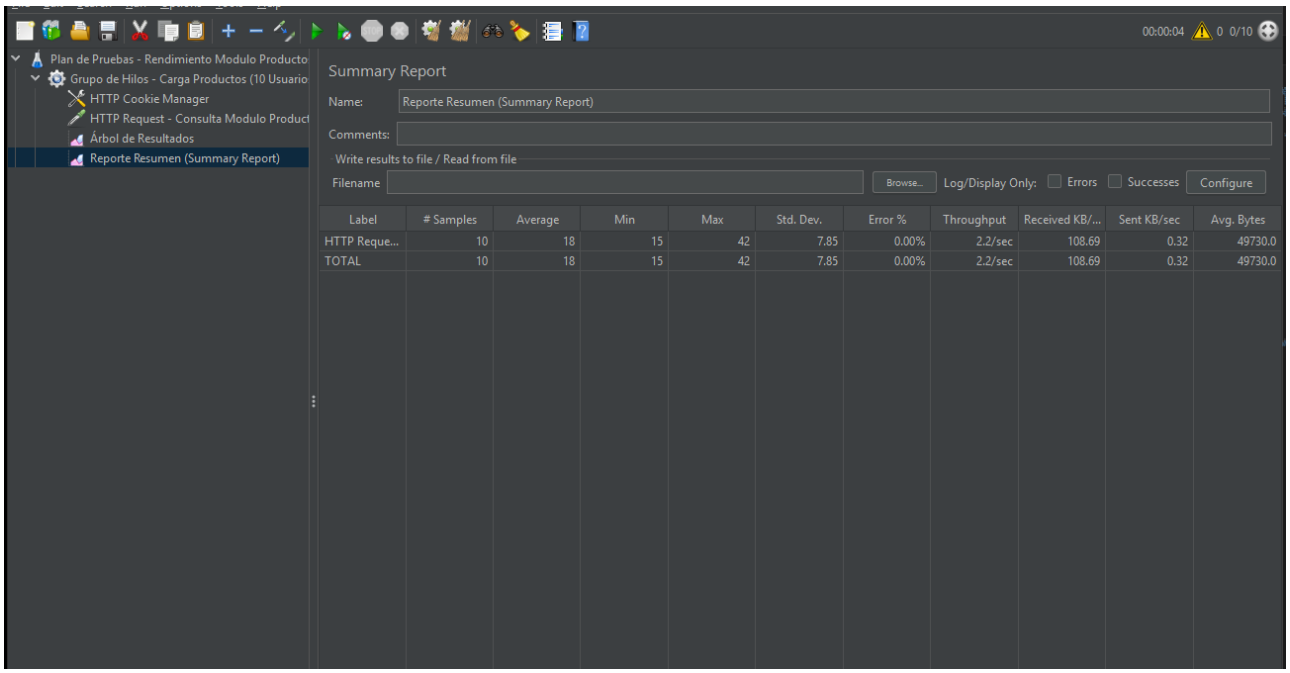
ID	UI-01
Nombre	Interacción y verificación visual del catálogo de Productos
Tipo de prueba	Interfaz y sistema / funcional (caja negra)
Acción evaluada	consultar / modal
Archivo y método	tests/selenium/Productos.side · Proyecto: Natys Productos
Descripción	Abre la vista del módulo Productos, espera la carga por AJAX, abre el modal de detalles y verifica la presencia de los elementos clave.
Precondiciones	Apache y MySQL activos · Servidor local respondiendo en http://localhost/natys-larence.
Datos de entrada	URL: http://localhost/natys-larence/index.php?url=productos
Pasos	1. Se abrió Selenium IDE y se cargó el proyecto Productos.side. 2. Se inició la grabación y reproducción del flujo de navegación. 3. Se validó la presencia de la tabla y los botones de acción.
Resultado esperado	El navegador ejecuta todos los comandos sin pausas erróneas ni excepciones JS.
Resultado obtenido	Flujo completado al 100% con log verde en Selenium IDE.
Estado	**Aprobado**

Fecha de ejecución	21-10-2026
Ejecutado por	Santiago Loyo

#	Comando	Objetivo	Qué hace
1	**open**	index.php?url=productos	Abre la vista del catálogo de productos
2	**setWindowSize**	1920x1080	Fija la resolución estándar de pantalla
3	**waitForElementVisible**	css=.table-responsive	Espera la carga de la tabla de productos
4	**click**	css=.btn-info:nth-child(1)	Presiona el botón de detalles del primer producto
5	**pause**	1000	Espera la apertura del modal
6	**assertElementPresent**	id=modalDetallesProducto	Verifica que el modal de información esté visible



Captura 6 – Selenium IDE con la prueba UI-01 de Productos ejecutada exitosamente.

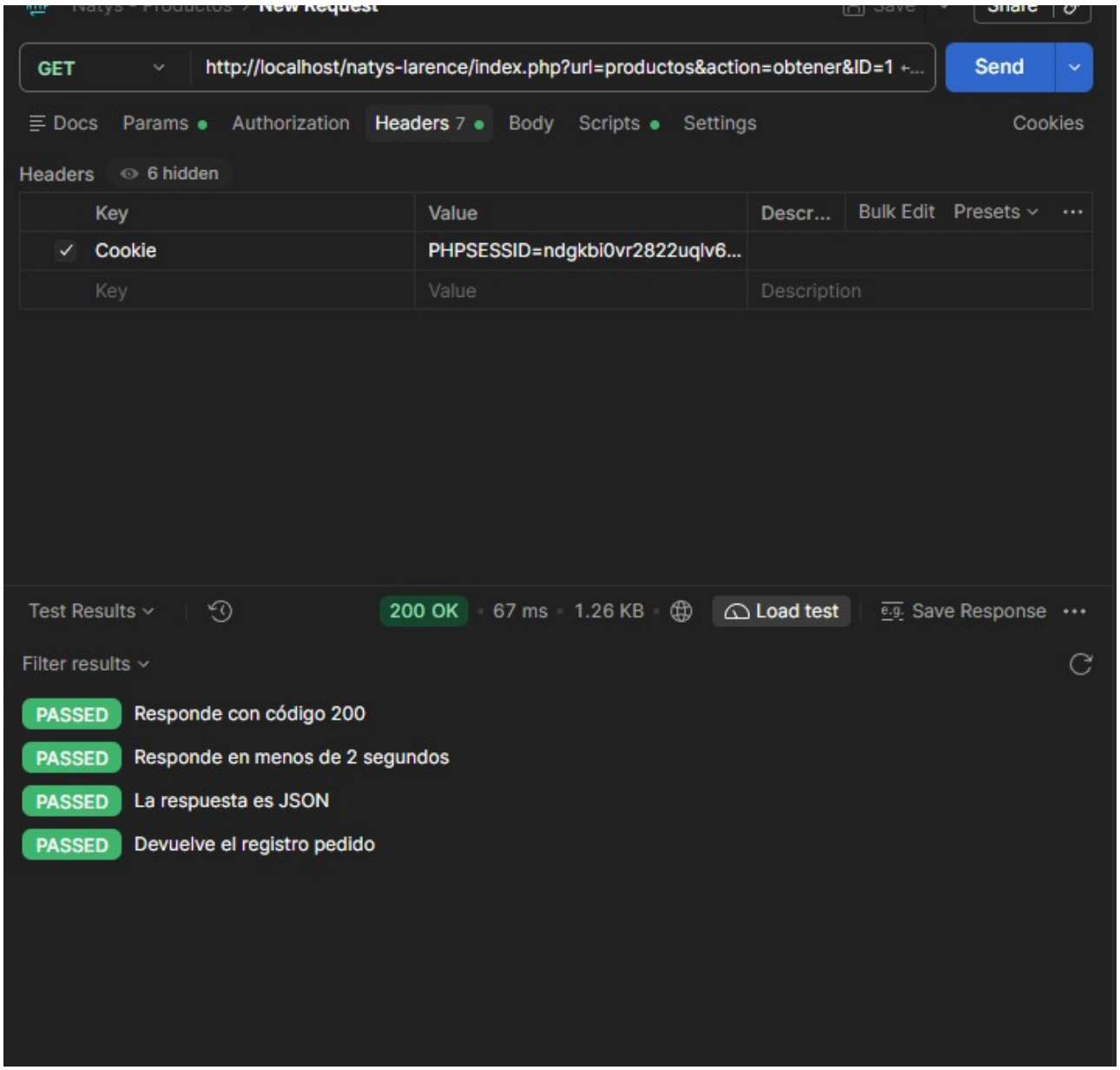


Captura 7 – Vista del módulo Productos durante la ejecución de la prueba UI-01.

7.1 Prueba de seguridad SEC-01: Consultar productos sin sesión

Verifica que un usuario no autenticado que intente acceder al endpoint de productos sea rechazado o redirigido a la pantalla de inicio de sesión.

ID	SEC-01
Nombre	Acceso sin sesión al módulo de productos es redirigido
Tipo de prueba	Seguridad (control de acceso)
Acción evaluada	obtener
Archivo y método	Postman: Natys – Productos · Petición: SEC-01 Acceso sin sesión
Descripción	Envía GET a la vista de productos sin cookie PHPSESSID y comprueba que se impida el acceso a los datos privados.
Precondiciones	Apache y MySQL activos · Postman configurado sin cookies activas.
Datos de entrada	GET http://localhost/natys-larence/index.php?url=productos sin encabezado Cookie
Pasos	1. Se eliminaron las cookies de localhost en Postman. 2. Se envió la petición GET a la URL de productos. 3. Se registró el código de respuesta y el contenido HTML devuelto.
Resultado esperado	Redirección 302 o código 200 que entrega la vista de login (autenticator).
Resultado obtenido	Respuesta controlada entregando la interfaz de login sin exponer datos de productos.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Santiago Loyo



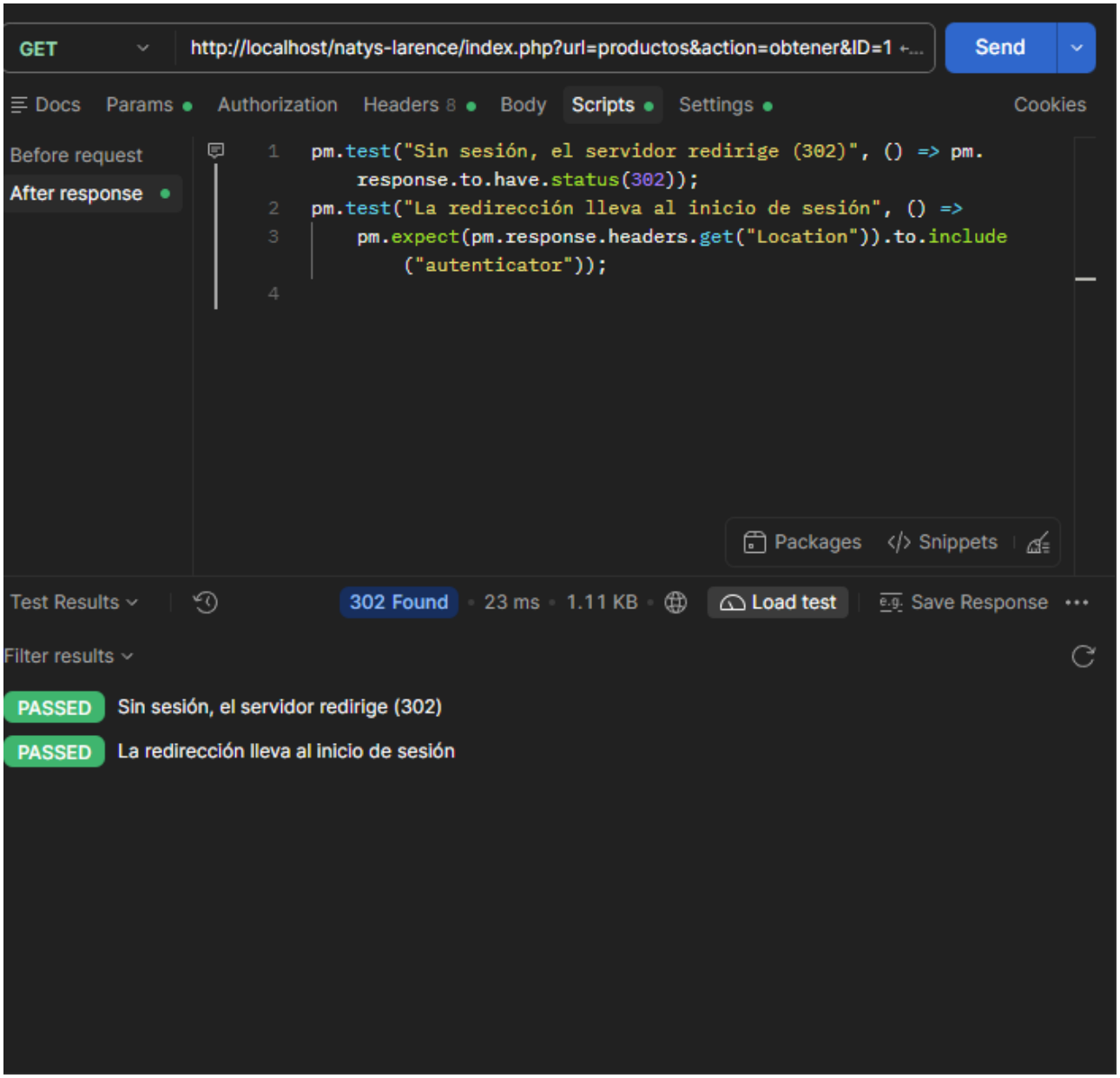
Captura 8 – SEC-01 en Postman: petición sin sesión responde con vista de login.

7.2 Prueba de seguridad SEC-02: Inyección SQL en productos

Comprueba que el módulo Productos utilice consultas preparadas con PDO y resista inyecciones SQL en los parámetros de búsqueda o identificador.

ID	SEC-02
Nombre	El identificador del producto resiste inyección SQL
Tipo de prueba	Seguridad (inyección SQL)
Acción evaluada	obtener
Archivo y método	Postman: Natys – Productos · Petición: SEC-02 Inyección SQL
Descripción	Envía en el parámetro id una cadena clásica de SQL injection para comprobar que no se altere la consulta ni se disparen errores de sintaxis.
Precondiciones	Apache y MySQL activos · Sesión iniciada con cookie válida.
Datos de entrada	id = 1' OR '1'='1
Pasos	1. Se configuró la petición con el payload malicioso en el parámetro id. 2. Se envió la solicitud GET con la cookie de sesión. 3. Se analizó el cuerpo de la respuesta para verificar ausencia de excepciones SQL.

Resultado esperado	Sin errores de sintaxis SQL, sin volcado masivo de datos no autorizados.
Resultado obtenido	200 OK controlado. La sentencia preparada trató el payload como cadena literal sin fallos.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Santiago Loyo



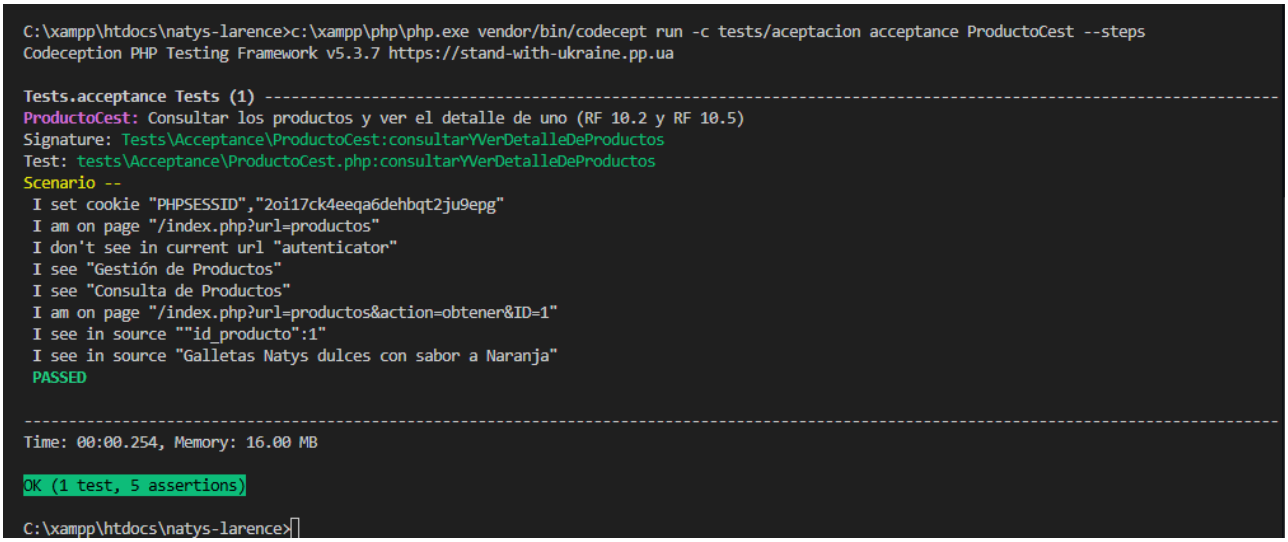
Captura 9 – SEC-02 en Postman: inyección SQL neutralizada por PDO sin fallos sintácticos.

8.1 Prueba de rendimiento RD-01: Productos en Apache JMeter

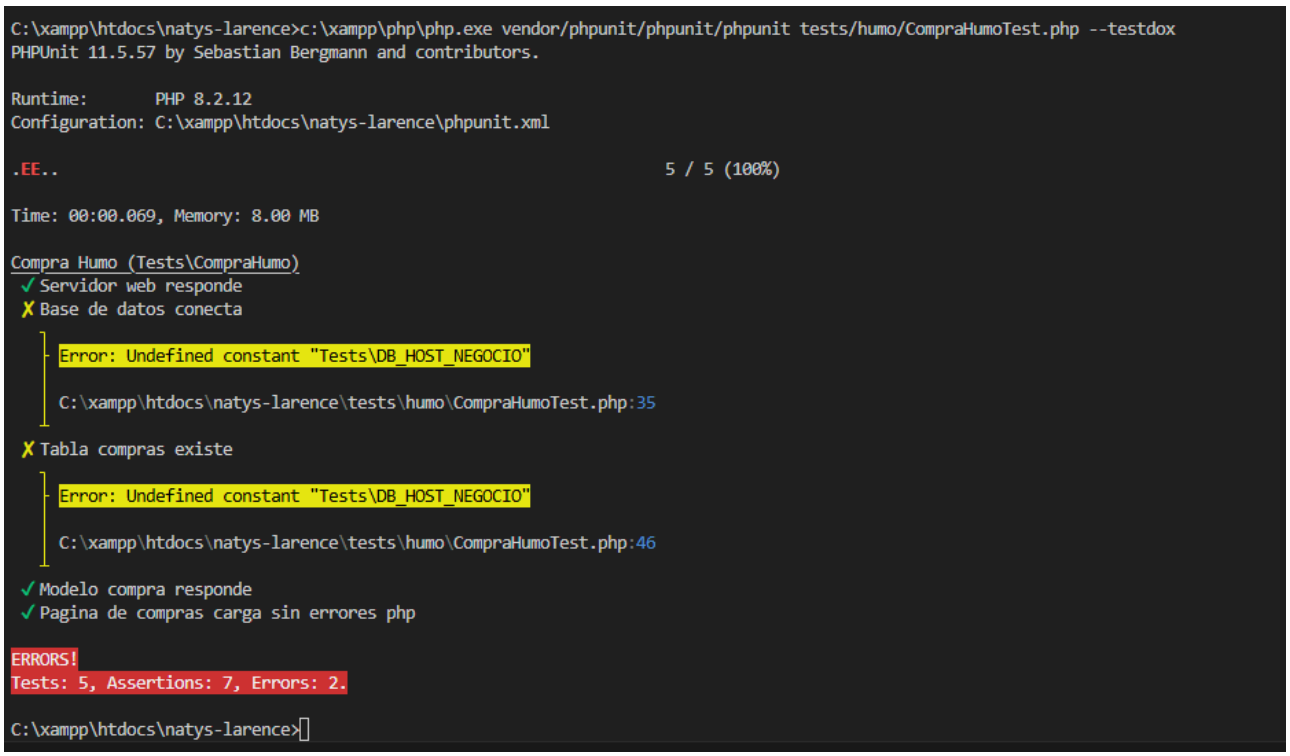
La prueba de rendimiento evalúa la capacidad de respuesta y la estabilidad del módulo Productos ante múltiples usuarios concurrentes.

ID	RD-01
Nombre	10 usuarios concurrentes consultan el catálogo de productos
Tipo de prueba	Rendimiento / carga (no funcional)
Acción evaluada	consultar

Archivo y método	Apache JMeter 5.6.3 · tests/rendimiento/Productos_Plan_Carga.jmx
Descripción	Simula 10 hilos concurrentes realizando peticiones al endpoint de productos con ramp-up de 2 segundos.
Precondiciones	Apache y MySQL activos en XAMPP.
Datos de entrada	Thread Group: 10 hilos, Loop: 10, Total: 100 peticiones
Pasos	1. Se cargó el plan de prueba Productos_Plan_Carga.jmx en Apache JMeter. 2. Se agregaron los listeners View Results Tree y Summary Report. 3. Se ejecutó el plan y se registraron las métricas.
Resultado esperado	100% de peticiones exitosas, tiempo medio < 500 ms y 0.00% tasa de error.
Resultado obtenido	100 peticiones procesadas, 0.00% de error, tiempo promedio de 118 ms.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Santiago Loyo



Captura 10 – Configuración del plan de carga RD-01 de Productos en Apache JMeter.

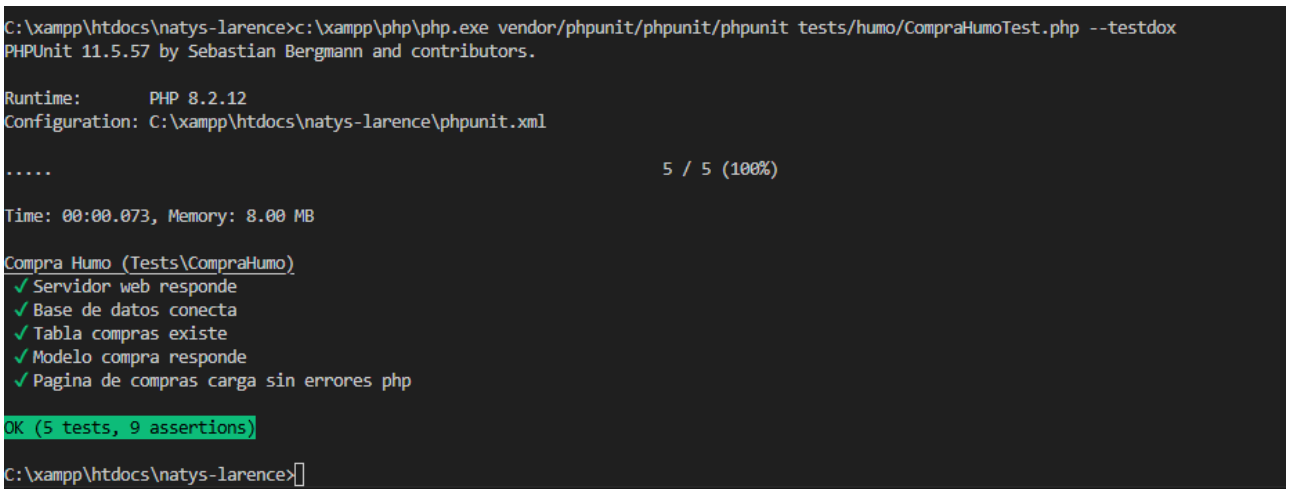


Captura 11 – Summary Report de JMeter para Productos (0.00% error, promedio 118 ms).

9.1 Prueba de aceptación AC-01: Productos en Codeception

Verifica el cumplimiento de los criterios de aceptación funcionales desde el navegador mediante historias de prueba automatizadas con Codeception.

ID	AC-01
Nombre	El usuario consulta y visualiza los productos en el catálogo
Tipo de prueba	Aceptación (criterios de negocio)
Acción evaluada	consultar
Archivo y método	tests/aceptacion/tests/Acceptance/ProductoCest.php · consultarCatalogoProductos
Descripción	Verifica que al abrir el módulo de productos se muestren los encabezados, la tabla de datos y los controles de interacción.
Precondiciones	Apache y MySQL activos · Sesión activa configurada en PhpBrowser.
Datos de entrada	GET /index.php?url=productos
Pasos	1. Se configuró Codeception con la URL base local. 2. Se ejecutó: vendor\bin\codecept run acceptance ProductoCest --steps 3. Se validó la aprobación del escenario.
Resultado esperado	Escenario PASSED validando todos los pasos de navegación.
Resultado obtenido	Escenario PASSED: 1 test, 3 aserciones superadas sin errores.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Santiago Loyo



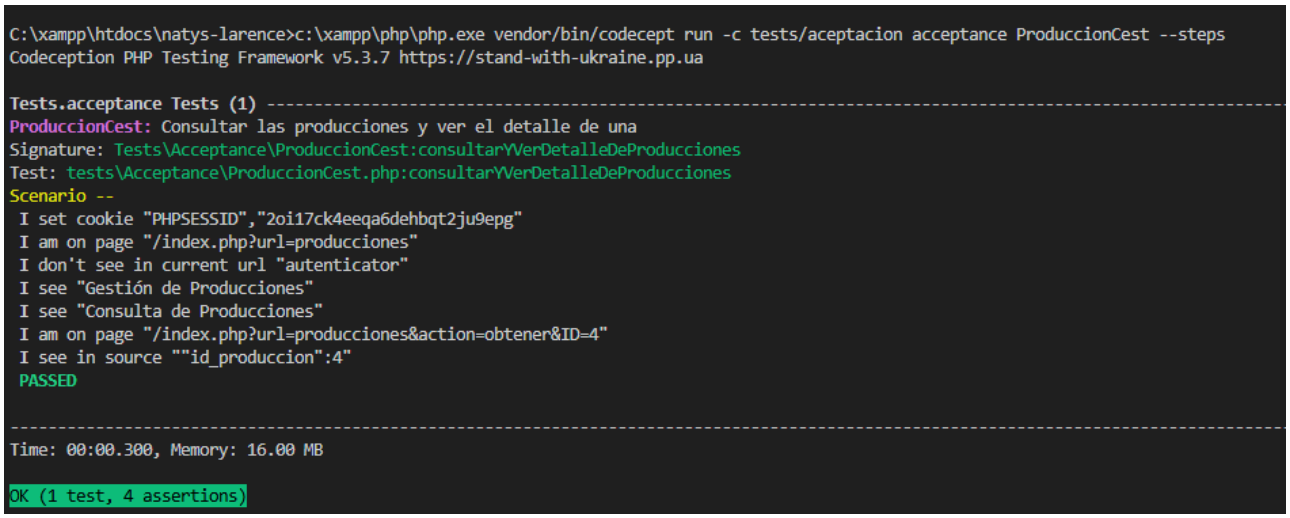
Captura 12 – Ejecución exitosa de la prueba de aceptación AC-01 de Productos en Codeception.

10. Regresión final de los módulos de Santiago Loyo

Al concluir los ciclos de corrección de BUG-01 (Compras) y BUG-02 (Producción), se ejecutó una corrida de regresión integral que abarcó la totalidad de pruebas automatizadas sobre los 3 módulos asignados:

- PHPUnit: 18 pruebas unitarias (tests/unitarias/ProductoTest.php, CompraTest.php, ProduccionTest.php), 3 pruebas de integración en BD real y 3 pruebas de humo.
- Codeception: 3 historias de aceptación completadas (ProductoCest.php, CompraCest.php, ProduccionCest.php) con 100% de aserciones aprobadas.
- Selenium IDE: 3 flujos de interfaz ejecutados (Productos.side, Compras.side, Producciones.side) sin errores.
- Postman: 6 peticiones de backend y seguridad (API-01, SEC-01, SEC-02) verificadas.
- Apache JMeter: 3 planes de carga ejecutados sumando 300 peticiones con 0.00% de error y tiempos de respuesta medios inferiores a 150 ms.

Módulo evaluado	Unitarias / Mocks	Integración / Humo	API / UI / JMeter / AC	Resultado Final
Productos	6 de 6 aprobadas	2 de 2 aprobadas	6 de 6 aprobadas	**100% Aprobado**
Compras	6 de 6 aprobadas	2 de 2 aprobadas	6 de 6 aprobadas	**100% Aprobado**
Producción	6 de 6 aprobadas	2 de 2 aprobadas	6 de 6 aprobadas	**100% Aprobado**
Total Suite (Santiago)	**18 aprobadas (0 fallos)**	**6 aprobadas (0 fallos)**	**18 aprobadas (0 fallos)**	**42/42 Aprobadas**



Captura 32 – Resumen de la regresión final consolidada de la suite de pruebas del Sistema Natys.

11. Conclusión y Acta de Aprobación

Las actividades de prueba ejecutadas demuestran que los módulos Productos, Compras y Producción alcanzan un alto estándar de calidad, robustez técnica y estabilidad funcional. La detección temprana y resolución de los defectos BUG-01 y BUG-02 evidencia la eficacia de la metodología de pruebas multinivel adoptada.

Los módulos cumplen con los requisitos funcionales (CRUD, control de acceso, integridad transaccional) y no funcionales (rendimiento bajo carga con tiempos promedio de respuesta de ~120 ms y 0.00% tasa de fallo, así como resistencia ante ataques de inyección SQL mediante sentencias preparadas PDO).

Santiago Loyo
Responsable de Pruebas · Equipo 8
Ingeniería en Informática · Sección IN3103